

if i want to develop a data model that i wish to store in mongodb as json-ld, should we utilize linkml and convert to json-ld for storage in mongo? In other words, is there value in working with linkml as the base format for modeling?

Recommendation

Yes—use LinkML as the authoritative modeling format, then generate the JSON-oriented artifacts used by MongoDB. For the HEW/NEXUS Data Commons, this provides substantially more value than designing directly in JSON-LD.

The pattern should be:

LinkML schema → JSON Schema + JSON-LD context → validated JSON/JSON-LD documents in MongoDB

LinkML becomes the **source of truth for the model**, while MongoDB remains the operational document store.

Why LinkML adds value

JSON-LD primarily supplies semantic interpretation: it maps JSON keys and values to globally identified concepts. It does **not**, by itself, fully define required fields, cardinalities, datatypes, permissible values, inheritance, or structural validation. LinkML can define both the document structure and its semantic mappings, then generate JSON Schema, JSON-LD contexts, SHACL, RDF, documentation, Python/Pydantic classes, and other artifacts from the same model.  Link... +2

That distinction matters for your use case:

Concern	LinkML	JSON-LD alone
Required and optional fields	Yes	No
Cardinality	Yes	No
Datatypes and ranges	Yes	Limited
Controlled vocabularies	Yes	Only semantic references

Concern	LinkML	JSON-LD alone
Ontology mappings	Yes	Yes
JSON Schema generation	Yes	No
SHACL/RDF generation	Yes	No
Human-readable documentation	Yes	No
Reusable application models	Yes	No

Your HEW-NEXUS design already calls for a modular metadata model, a minimum metadata profile, metadata validation services, ECTO-centered semantics, provenance, persistent identifiers, access policies, and links among datasets, workflows, cohorts, outputs, and publications. Those requirements are a strong fit for a schema-first approach such as LinkML.

Do not think of it as “converting LinkML into JSON-LD for storage”

A better conceptual model is:

- 1. Author the model in LinkML.**
- 2. Generate a JSON Schema** for structural validation.
- 3. Generate a JSON-LD context** for semantic interpretation.
- 4. Store ordinary MongoDB-friendly documents** that conform to that structure.
- 5. Attach or reference the JSON-LD context** so the same documents can be interpreted as linked data.

For example, a stored document could look like:

JSON

```
{
  "@context": "https://example.org/hew/context/v1.jsonld",
  "@id": "https://example.org/dataset/epa-airnow-2026",
  "@type": "Dataset",
  "title": "AirNow PM2.5 Observations",
  "exposureConcept": {
    "@id": "http://purl.obolibrary.org/obo/ECTO_..."
  },
  "spatialCoverage": {
    "type": "UnitedStates"
  },
  "accessLevel": "open",
```

```
"schemaVersion": "1.0.0"  
}
```

This is simultaneously:

- a normal queryable MongoDB document;
- JSON conforming to your generated structural schema;
- JSON-LD when processed with the referenced context.

You generally do **not** need to expand every field into full IRIs before storing it. Compact JSON-LD is usually more usable for application developers and MongoDB queries.

Recommended architecture

1. LinkML repository

Maintain version-controlled LinkML modules such as:

```
hew-model/  
  core.yaml  
  dataset.yaml  
  study.yaml  
  cohort.yaml  
  exposure.yaml  
  geospatial.yaml  
  workflow.yaml  
  provenance.yaml  
  access_policy.yaml  
  mappings/  
    ecto.yaml  
    schema_org.yaml  
    dcat.yaml
```

Use multiple modules rather than one large schema. This aligns with the modular Data Commons design in your ecosystem summary.

2. Generated artifacts

From the LinkML source, publish:

```
dist/  
  hew.schema.json  
  hew.context.jsonld  
  hew.shacl.ttl  
  hew.owl.ttl  
  hew-model-docs/  
  python/  
  typescript/
```

LinkML explicitly supports generation of JSON-LD contexts from a schema, allowing the same model to control both JSON shape and RDF interpretation.

 Link... +1

3. Validation pipeline

Use LinkML validation in the ingestion or CI pipeline:

```
graph TD
    A[Incoming record] --> B[LinkML validation]
    B --> C[Vocabulary and identifier checks]
    C --> D[MongoDB-compatible validation]
    D --> E[MongoDB]
```

The application-level LinkML validator should be the primary, complete validator.

MongoDB collection validation can provide a second guardrail, but the generated LinkML JSON Schema will probably need a small transformation step. MongoDB supports JSON Schema draft 4 with extensions and omissions; for example, it uses BSON-specific types and does not support `$ref`, `definitions`, `format`, or the standard `integer` type in the same way.  Mong... +1

Therefore:

Do not assume that a LinkML-generated JSON Schema can be installed unchanged as a MongoDB `$jsonSchema` validator.

Generate a flattened Mongo-specific validator or maintain a lightweight projection containing the most important database constraints.

4. MongoDB documents

Store:

- stable identifiers;
- explicit model and record versions;
- compact ontology identifiers or CURIEs;
- provenance;
- access classification;
- spatial and temporal metadata;
- links to related resources;

- optionally, `@context` and `@type`.

Suggested administrative fields:

```
JSON

{
  "_id": "dataset:epa-airnow-2026",
  "@id": "https://example.org/dataset/epa-airnow-2026",
  "@type": "Dataset",
  "schemaVersion": "1.2.0",
  "recordVersion": 3,
  "createdAt": "2026-07-01T14:00:00Z",
  "modifiedAt": "2026-07-20T18:30:00Z",
  "provenance": {
    "source": "EPA",
    "ingestionWorkflow": "workflow:airnow-import-v2"
  }
}
```

Using the same value for MongoDB `_id` and the semantic identifier is possible, but not required. Frequently it is cleaner to retain:

- `_id` for MongoDB mechanics;
- `@id` or `id` for the globally stable identifier.

Where LinkML is especially useful for HEW

LinkML would let you explicitly model distinctions such as:

```
Dataset
├─ EnvironmentalLayer
├─ SensorStream
├─ CohortDataset
├─ SurveyDataset
└─ ResultBundle
```

and reusable components such as:

```
SpatialCoverage
TemporalCoverage
AccessPolicy
DataUseCondition
Provenance
QualityStatement
OntologyMapping
Distribution
ResponsibleAgent
```

For exposure information, I would not make an ECTO class identifier the entire exposure record. Use ECTO as the **exposure concept spine**, surrounded by an observation or derived-feature model containing:

```
exposure_concept
agent
receptor
route
medium
place
time_window
measurement_value
unit
derivation_method
uncertainty
source_dataset
provenance
```

That corresponds closely to the HEW-NEXUS recommendation to pair ECTO with an exposure-event or exposure-observation schema rather than use ECTO alone.

Important caveats

LinkML's direct JSON-LD generator is not the part I would rely on most

The current LinkML documentation labels direct generation of a JSON-LD representation of the **schema** as incompletely implemented. The JSON-LD **context generator**, however, is the more relevant artifact for making instance JSON semantically interpretable. [LinkML context generator](#) +1

So the dependable workflow is:

```
LinkML model
├─ generate JSON Schema
├─ generate JSON-LD context
└─ serialize application records as JSON
    +
    apply/reference context
```

rather than relying heavily on `gen-jsonld` to generate every production representation.

MongoDB will not become an RDF graph database

MongoDB can store JSON-LD, but its native indexes and queries operate over BSON document structure, not RDF graph semantics. You will not automatically gain:

- SPARQL;
- ontology reasoning;
- subclass inference;
- transitive property reasoning;
- graph-wide SHACL validation.

For those functions, create an export or synchronization path:

```
MongoDB JSON-LD
    ↓
JSON-LD expansion/RDF conversion
    ↓
Knowledge graph or triple store
```

This is a good fit for later Biolink, Monarch, or ROBOKOP-oriented knowledge integration without forcing the operational catalog to run as a triple store.

When LinkML may be unnecessary

LinkML may be excessive when all of the following are true:

- the schema is very small;
- it is application-specific;
- it will never be exchanged outside one service;
- no ontology mappings are required;
- no alternate representations are needed;
- no community governance or formal versioning is anticipated.

Those conditions do not appear to apply to the HEW Data Commons. Your requirements involve multiple resource types, FAIR discovery, ontologies, geospatial standards, governed assets, workflows, and possible knowledge-graph reuse. In that setting, a technology-neutral canonical model has high value.

Bottom line

Use LinkML as the canonical model, not as the MongoDB storage syntax.

The practical recommendation is:

```
Canonical model:      LinkML
Structural interchange: JSON
```

Semantic layer:	Generated JSON-LD context
Application validation:	LinkML validator / generated JSON Schema
Database validation:	MongoDB-specific reduced JSON Schema
Operational storage:	MongoDB BSON documents
Knowledge-graph export:	JSON-LD/RDF, SHACL and ontology mappings

This gives you a single governed model while preserving MongoDB's flexibility and leaving a credible path to ECTO, DCAT, Schema.org, FHIR, GA4GH, OGC, Biolink, Monarch, and other semantic ecosystems.